

What the LO fast grid computes

Notes on `fastgrid_codes/sf_lo_{L,T}_fastgrid.py`

2 October 2026

In one sentence. The `make` mode still does the z -integral at fixed r , exactly as before; it then turns those values into one number w_n per node r_n of the dipole table, so that the `use` mode is the dot product $F = \sum_n w_n N(r_n, Y)$ of the weights with one row of the table.

1 The integral to be computed

With $u = \ln r$, the LO structure functions are a single integral over the dipole size,

$$F_{L,T}(x, Q^2) = \int du G_{L,T}(u) N(u, Y), \quad Y = \ln \frac{x_0}{x}, \quad (1)$$

where all the dependence on the photon is in the kernel G , which is the **z -integral at fixed r** :

$$G_L(u) = \sum_f \frac{2N_c Q^4 e_f^2}{\pi^3} r^2 \int_0^1 dz z^2 (1-z)^2 K_0^2(\epsilon_f r), \quad (2)$$

$$G_T(u) = \sum_f \frac{N_c Q^2 e_f^2}{2\pi^3} r^2 \int_0^1 dz \left\{ [z^2 + (1-z)^2] \epsilon_f^2 K_1^2(\epsilon_f r) + m_f^2 K_0^2(\epsilon_f r) \right\}, \quad (3)$$

with $r = e^u$, $\epsilon_f^2 = z(1-z)Q^2 + m_f^2$ and a factor $\sigma_0/2$ left out. One factor r comes from $d^2r = 2\pi r dr$ and the other from $dr = r du$. G depends on Q^2 but not on x or on the dipole; that is what makes a fast grid possible. In the code, G at a given r is `prefactor * r**2 * z_integral_{L,T}(r, ...)`.

2 What we know about the dipole

The dipole is only known at the nodes of its table, which are uniformly spaced in u :

$$u_n = \ln r_{\min} + n h, \quad h = \ln(\text{mult}), \quad N_n(Y) \equiv N(u_n, Y), \quad n = 0, \dots, n_r - 1. \quad (4)$$

Between the nodes, the Julia and Python reference codes (`sf_lo_{L,T}.jl`, `sf_lo_{L,T}.py`) interpolate linearly in u . That interpolant can be written with the “hat” functions ϕ_n of Fig. 1:

$$N(u, Y) = \sum_n N_n(Y) \phi_n(u), \quad \phi_n(u) = \max\left(0, 1 - \frac{|u - u_n|}{h}\right). \quad (5)$$

Each ϕ_n is 1 at its own node, 0 at all other nodes, and linear in between.

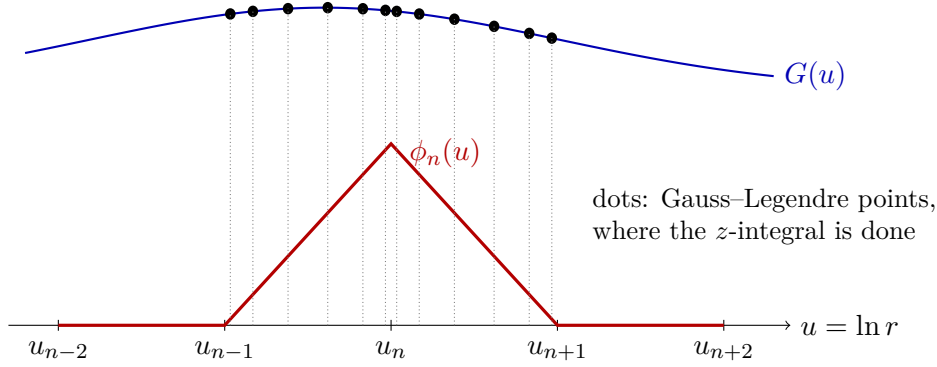


Figure 1: Uniform grid in $u = \ln r$ (the dipole table's own nodes), the hat function ϕ_n of node n , and the kernel $G(u)$. The weight w_n is the area under $G \phi_n$; it is computed from G at the marked points in the two intervals next to u_n .

3 From the integral to a dot product

Inserting Eq. (5) into Eq. (1) gives

$$F(x, Q^2) = \sum_n w_n N_n(Y), \quad w_n = \int du G(u) \phi_n(u) \quad (6)$$

This is exact for the linearly interpolated dipole: no further approximation is made. The weights w_n live on exactly the dipole's grid and do not depend on x or on the dipole, so

- `make` computes the vector $(w_0, \dots, w_{n_r-1})$ once per Q^2 (and polarisation) and writes it to the grid file, one row per node;
- `use` reads one row of the dipole table, $N_n(Y)$, and returns $\sum_n w_n N_n(Y)$. If $Y = \ln(x_0/x)$ falls between two table rows, the row is first interpolated linearly in Y , as in the reference codes; if it falls on a row, it is literally that row.

4 Where the z -integral at fixed r comes in

Written out, the weight is a weighted average of G over the two intervals next to the node:

$$w_n = \int_{u_{n-1}}^{u_n} du G(u) \frac{u - u_{n-1}}{h} + \int_{u_n}^{u_{n+1}} du G(u) \frac{u_{n+1} - u}{h}. \quad (7)$$

These are ordinary one-dimensional integrals over u of a smooth function. `make` evaluates them with a 6-point Gauss-Legendre rule on each interval $[u_k, u_{k+1}]$ (exact for polynomials up to degree 11):

$$\int_{u_k}^{u_{k+1}} du f(u) \simeq \frac{h}{2} \sum_{j=1}^6 \omega_j f\left(u_k + \frac{h}{2}(1 + t_j)\right), \quad (8)$$

with t_j, ω_j the Legendre nodes and weights. **Every evaluation of G at such a point is the z -integral at fixed $r = e^u$, done by SciPy's `quad` exactly as before.** So the z -integral at fixed r is still the basic step; it is just done at six values of r inside each node interval rather than only at the nodes, and the results are then combined into one number per node. Gauss-Legendre is only the rule for the u -integral that builds w_n ; it does not change the grid, which is the dipole's.

To lowest order $w_n \simeq h G(u_n)$, since $\int \phi_n du = h$. The difference from $h G(u_n)$ accounts for how G varies across the hat.

Ends of the grid. Below the first node the dipole is taken to be zero. If the upper limit `--rmax` lies beyond the last node, the dipole is held at its last value there, as in the reference codes, and that tail is added to the last weight. If `--rmax` lies inside the grid, only the part of each hat below it is integrated.

5 The three ways of building the weights

All three give F as a sum over grid points; they differ in which r values G is evaluated at and in what is assumed about the dipole between its nodes.

	G evaluated at	use step	Accuracy
(a) Previous version: fine Simpson grid	2001 points uniform in u , independent of the dipole	interpolate N onto the fine grid, then Simpson sum	error from the kinks of the linear interpolant at the table nodes; needed spacing $\ll h$; error $\sim 10^{-6}$ at 2001 points
(b) Node quadrature: $w_n = s_n h G(u_n)$	the nodes only	dot product with the table row	$\mathcal{O}(h^4)$ for a <i>smooth</i> dipole through the nodes, but differs from the linear-interpolation result by $\mathcal{O}(h^2)$
(c) Current: hat weights, Eq. (6)	6 Gauss-Legendre points per node interval	dot product with the table row	exact for the linearly interpolated dipole; matches the reference codes

Here s_n are quadrature weights, for example the extended Simpson weights that `BK_evolve_tf.py` uses,

$$(s_0, s_1, s_2, s_3, \dots, s_{n_r-1}) = (9, 28, 23, 24, \dots, 24, 23, 28, 9)/24.$$

Options (b) and (c) disagree by the error of linear interpolation itself, which is of order $h^2 N''$. Your BK solver documents this effect: `BK_evolve_tf.py` switched to cubic splines because linear interpolation of the convex $N(\ln r)$ biased every evaluation high by about +0.08% on its grid. So the choice between (b) and (c) is a choice of what the dipole is between its nodes:

- (c) reproduces the Julia and Python reference codes, which interpolate linearly;
- (b) treats the tabulated dipole as samples of a smooth function, as your BK solver does internally, and needs G only at the nodes.

Both give a dot product with a row of the table on the same uniform grid. Switching from (c) to (b) is a small change in `make`.

6 Checks of the current version

- Against a direct high-precision integral of G times the linearly interpolated dipole (adaptive, interval by interval, relative tolerance 10^{-11}), for `median_bk` at $Q^2 = 10 \text{ GeV}^2$, $x = 10^{-3}$, $r \leq 20 \text{ GeV}^{-1}$: agreement to 2×10^{-12} (F_L) and 3×10^{-13} (F_T).
- Against the Julia reference codes at six points (both polarisations, both dipole tables, $Q^2 = 2$ to 27 GeV^2 , with and without charm), using `--rmax 20` to match their `xmax`: agreement to

7.4×10^{-6} (1.2×10^{-5} for one charm part), which is the Julia codes' own quadrature tolerance of 10^{-5} .